

Proofs and Prices: Machine-Checked Soundness and Measured Trade-offs in a Minimalist Chess Engine

Thomas Dybdahl Ahle Second Author

Draft of August 5, 2026

Abstract

Sunfish is a chess engine whose whole logic fits in 131 lines of Python, and whose docstrings state, in unusually precise English, the contracts its search is supposed to satisfy. We report on a one-day campaign that treated those docstrings as latent formal specifications: we transcribed them into Lean 4, proved the ones that are true, refuted the ones that are not, and named the ones that are only conditionally true — and, in parallel, priced every resulting engine change with ELO tournaments under a fixed empirical methodology. The formal deliverables include a sorry-free proof of the engine’s core fail-soft null-window contract, a machine-checked seven-position counterexample showing the engine’s stalemate correction is sound only under a named hypothesis, a same-day proof of a maintainer-conjectured invariant about killer moves, and an interval-valued specification for Late Move Reductions together with a machine-checked transposition-table inconsistency it induces. These results organize into a soundness taxonomy for selective search: the search window may shape termination, and may trigger shortcuts whose value provably one-side-bounds the same value function; it must never select between incomparable evaluations. Each unproven assumption is kept as a named hypothesis — a ledger of bets — and, where possible, measured on the production engine, which plays publicly on Lichess. Proofs identify the trade; tournaments price it.

1 Introduction

Sunfish¹ is a minimalist chess engine: with comments and whitespace removed, the engine proper is 131 lines of Python, yet it plays at ratings above 2000 on Lichess, where it runs in production as the bot `sunfish-engine`. What makes it an unusual verification target is not its size but its *documentation*: the docstring of its central routine, `Searcher.bound`, is a complete two-sided fail-soft contract, stated a decade before anyone tried to prove it. The comments scattered through the search — “`lower <= s(pos) <= upper`”, “the opponent will for sure just stand pat”, “sunfish may report ‘mate’, but then after more search realize it’s not a mate after all” — turn out to be, respectively: a table invariant, a dischargeable lemma, and the exact failure mode of a hypothesis that a machine-checked counterexample shows to be necessary. The docstrings were latent formal specs.

This paper reports a campaign, conducted in a single day on the live repository, that ran two loops in parallel:

- **The proof loop.** Every search feature was transcribed into a small, dependency-free Lean 4 model (core Lean plus the `omega` tactic; no Mathlib; a full build takes seconds)

¹<https://github.com/thomasahle/sunfish>

and either proven sorry-free, refuted by an explicit machine-checked counterexample, or reduced to a *named hypothesis* whose statement — not whose proof — is the deliverable.

- **The price loop.** Every engine change motivated by the proofs (and every change the proofs advised against) was priced by an ELO tournament between frozen engine builds, under the methodology of Section 7, on the same engine that plays real games on Lichess.

The two loops answer different questions and neither substitutes for the other. A proof tells you *what* a pruning trick costs in soundness — which invariant it weakens, at which positions, under which window. A tournament tells you what that weakening costs (or earns) *in ELO*. Several of our measured numbers (Table 1) show soundness restorations that are strictly ELO-negative, and unsound-in-the-point-sense optimizations that are strongly ELO-positive; the formal work is what lets us merge the latter with our eyes open, with a two-line mitigation for the specific inconsistency the proof predicts.

Contributions. (1) A sorry-free Lean 4 proof that sunfish’s fail-soft null-window search satisfies its own docstring (Section 2) — to our knowledge the first Lean 4 formalization of null-window/MTD-bi search. (2) A faithful model of the engine’s king-capture convention and stalemate correction, with a proof under four individually necessary hypotheses and a machine-checked counterexample showing the key hypothesis cannot be dropped (Section 3). (3) A same-day proof of a maintainer-conjectured invariant: the stored killer move at a king-capturable position is itself a king capture (Section 4). (4) A soundness taxonomy for selective search (Section 5), with futility pruning and Late Move Reductions as the canonical citizens of its two nontrivial classes. (5) The ledger-of-bets discipline (Section 6) and the empirical methodology with honest numbers, including artifacts we initially believed (Section 7).

2 The Core Contract

2.1 The specification, transcribed verbatim

The docstring of `Searcher.bound` (sunfish.py, lines 286–290) reads:

```
Let s* be the "true" score of the sub-tree we are searching.
The method returns r, where
if gamma > s* then s* <= r < gamma (A better upper bound)
if gamma <= s* then gamma <= r <= s* (A better lower bound)
```

The model is an abstract game: a type of positions, a successor function $\text{moves} : \text{Pos} \rightarrow \text{List Pos}$ where the empty list means the side to move has lost, and a static evaluation $\text{eval} : \text{Pos} \rightarrow \mathbb{Z}$ (`Game, GameTree.lean`). The “true score” s^* is depth-limited negamax, $\text{negamax}_d(p) : \text{eval}(p)$ at depth 0, otherwise the maximum over moves m of $-\text{negamax}_{d-1}(m)$, folded from $\text{LOSS} = -M_U$ (`negamax, GameTree.lean`). Here $M_U = 69290$ and $M_L = 50710$ are sunfish’s `MATE_UPPER` and `MATE_LOWER`.

Definition 2.1 (`BoundSpec`, the docstring). For a game G , depth d , position p , window γ and report r ,

$$\text{BoundSpec}(G, d, p, \gamma, r) \equiv (\gamma \leq r \rightarrow r \leq \text{negamax}_d(p)) \wedge (r < \gamma \rightarrow \text{negamax}_d(p) \leq r).$$

(`BoundSpec, Bound.lean`)

The search itself, stripped to its logical skeleton, is the fail-soft loop with the early cutoff on $best \geq \gamma$, over children searched with the flipped null window $1 - \gamma$ one ply shallower (`bound`, `searchMoves`, `Bound.lean`). No table, no null move, no quiescence, no killer: those return in Sections 3–6.

Theorem 2.2 (The docstring is true). *For every game G , depth d , position p and window $\gamma \in \mathbb{Z}$, $\text{BoundSpec}(G, d, p, \gamma, \text{bound}_d(p, \gamma))$ holds. (`bound_spec`, `Bound.lean`)*

Proof sketch. Induction on depth. The loop is handled by a single invariant lemma (`searchMoves_spec`, `Bound.lean`): *the running best is itself a fail-soft-correct report of the running true maximum.* On a cutoff, $best \geq \gamma$ came either from the current child (whose report is $\leq$ its true value by the induction hypothesis) or from the previous $best$ (bounded by the invariant); either way the fold over the unsearched tail can only grow. Without a cutoff, both the child and the previous $best$ failed low, so both upper-bound their true counterparts and the invariant is re-established. The null-window step is the integer fact $b < 1 - \gamma \iff \gamma \leq -b$: a child failing low against $1 - \gamma$ fails high for the parent, and vice versa. The proof is sorry-free; `#print axioms` reports only `propext` and `Quot.sound`. $\square$

2.2 Why this is the load-bearing spec: the MTD-bi driver

Sunfish’s driver `Searcher.search` is MTD-bi (a bisecting variant of $\text{MTD}(f)$ [8]): it maintains a bracket $lower \leq score \leq upper$ and repeatedly probes `bound` at a γ inside the bracket. Theorem 2.2 is exactly the property the binary search needs: *every* probe, whatever its γ , moves one end of the bracket — a fail high raises *lower*, a fail low lowers *upper*, and the two can never cross (Theorem 5.5 below makes the no-crossing claim formal, and Section 5 shows how Late Move Reductions break precisely it).

3 The King-Capture Model and the Stalemate Correction

Sunfish generates *pseudo-legal* moves and scores an actual king capture as an immediate win: a position whose static score is $\leq -M_L$ has already lost its king and is normalized to exactly $-M_U$. “No legal moves” (mate *or* stalemate) therefore surfaces in the search as $best = -M_U$ after the loop, and the engine post-processes it (at depth > 2) with an in-check probe on the null-moved position: $best := -M_L$ if in check, else 0.

Definition 3.1 (Draw-aware value). $\text{negamaxDraw}_d(p)$ is negamax over the king-capture-normalized game with the depth-gated stalemate correction applied to the fold: if $d > 2$ and the fold equals *LOSS*, the value becomes $-M_L$ if p is in check and 0 otherwise. (`negamaxDraw`, `drawFix`, `Stalemate.lean`)

Definition 3.2 (CheckProbeOK). A probe function agrees pointwise with the one-ply king-capture reading of check: after passing the turn, the opponent has a move reaching a position with $\text{eval} \leq -M_L$. For the real engine this is the assumption that the depth-0 QS probe `bound(flipped, MATE_UPPER, 0) == MATE_UPPER` decides check — true because quiescence never prunes king captures. (`CheckProbeOK`, `inCheckB`, `Stalemate.lean`)

Named Hypothesis 3.3 (MateValuesAreKingCaptures). Whenever $\text{negamaxDraw}_d(p) = M_U$ with $d \geq 1$, some move of p actually reaches a position with $\text{eval} \leq -M_L$: mate-band sentinel values always come from a real king capture, never from a shallow-horizon artifact. (`MateValuesAreKingCaptures`, `Stalemate.lean`)

Theorem 3.4 (The corrected search satisfies the docstring). *Let G be a game with a pass move, and suppose: (i) all evaluations lie in $[-M_U, M_U]$ (Bounded); (ii) the probe satisfies CheckProbeOK; (iii) the window is in band: $-M_U < \gamma \leq M_U$ (an interval closed under the null-window flip $\gamma \mapsto 1 - \gamma$); and (iv) MateValuesAreKingCaptures. Then the stalemate-corrected search `boundStale` satisfies `BoundSpec` against `negamaxDraw` at every depth and position. (`boundStale_spec`, `Stalemate.lean`)*

Proof sketch. The same loop invariant as Theorem 2.2, plus a pointwise lemma (`staleFix_spec_core`, `Stalemate.lean`) showing the correction maps a fail-soft-correct pair (search *best*, true fold) to a fail-soft-correct pair. The crux is the masking step: if the *true* fold is *LOSS*, then (by the band, the sentinel invariant, and Hypothesis 3.3) every child is an exact king-capture mate, every child report is exactly *LOSS*, and hence the search’s *best* is *LOSS* too — so both sides apply the *same* correction, using the correct probe. In-band windows are needed so that a fail-low at *LOSS* certifies the loop ran to completion: a $\gamma \leq -M_U$ would allow an early cutoff at *LOSS* with unsearched moves remaining. Sorry-free. $\square$

Each hypothesis is individually necessary; the interesting one is (iv):

Counterexample 3.5 (The correction is not unconditionally sound). There is a seven-position game with bounded honest evaluations, a perfectly correct probe, and an in-band window $\gamma = -5$, on which the faithful depth-gated correction returns a fail-low “upper bound” of -20 while the draw-aware value is 0 . The mechanism: two positions are terminal stalemates; seen at remaining depth ≤ 2 (gate off) they score *LOSS*, so their parents at depth 2 score a fabricated M_U — mate scores with no king capture anywhere behind them — and the root inherits a wrong bound. This is sunfish’s own code comment (“sunfish may report ‘mate’, but then after more search realize it’s not a mate after all”) turned into a machine-checked refutation. (`boundStale_not_unconditional`, `Cex`, `cex_violates_hypothesis`, `Stalemate.lean`)

Theorem 3.6 (Depth inconsistency is intrinsic). *The depth > 2 gate makes the draw-aware value itself depth-dependent: in the counterexample game there is a stalemate position g with $\text{negamaxDraw}_2(g) = \text{LOSS}$ and $\text{negamaxDraw}_3(g) = 0$. No depth-independent game value can agree with `negamaxDraw` at every depth. (`negamaxDraw_depth_inconsistent`, `Stalemate.lean`)*

Remark 3.7 (Why mate is $-M_L$, not $-M_U$). $-M_U$ is the reserved sentinel meaning “the king is *actually* capturable/gone”. Scoring checkmate-by-correction as $-M_U$ would re-inject the sentinel at positions whose king is not capturable and destroy the parent-level detection one ply up; it would also order “mated next move” above “king already lost”. The engine’s choice of $-M_L$ at line 412 is what keeps the sentinel trustworthy.

4 The Killer Invariant: Conjectured at Noon, Proved by Evening

While modeling the stalemate correction we recorded a potential gap: the killer move (the stored `tp_move` entry) is yielded *before* the sorted moves, so a non-capture killer failing high could let `bound` return a value below M_U at a king-capturable position — breaking the sentinel requirement that Theorem 3.4 leans on. The maintainer conjectured this cannot happen in sunfish. The conjecture was proved the same day.

Definition 4.1 (KillerOK, the table invariant). A killer table t (a partial map from positions to moves) satisfies KillerOK if every stored entry is a legal move of its key position, and at

a king-capturable position the stored entry — if any — is *itself a king capture*. (`KillerOK`, `Killer.lean`)

Theorem 4.2 (`KillerIsKingCapture`). *Thread `tp_move` through the search as state (position-keyed, store-on-fail-high-only), with the move loop running over value-ordered moves (king captures, value $\geq M_L$, strictly first). Then for every depth d , position p , in-band window $-M_U < \gamma \leq M_U$, and table t satisfying `KillerOK`:*

1. *the call `boundKilld(p,  $\gamma$ , t)` returns a table satisfying `KillerOK`; and*
2. *if $d \geq 1$, p ’s own king is not already gone, and p is king-capturable, the returned value is exactly M_U — on the killer path and on the loop path alike.*

(`boundKill_spec`, `Killer.lean`)

Proof sketch. Induction on depth, cases on the table lookup. *Killer-cutoff case:* the stored move is the killer itself; by the invariant it is a king capture whenever p is capturable, the king-capture normalization scores it at the full sentinel, and an in-band γ makes the sentinel fail high — so the cutoff both stores a king capture and reports exactly M_U . *Killer-fail-low case:* a king-capture killer would have reported $M_U \geq \gamma$, so p is not capturable and plain loop preservation applies. *No-killer case:* at a capturable position the capture heads the ordered list, cuts off immediately at the sentinel, and nothing else can be stored first (`killLoop_capture_head`, `Killer.lean`). Applied along any execution history starting from the empty table (`killerEmpty_OK`, `Killer.lean`), the invariant holds forever. Sorry-free; axioms `propext`, `Quot.sound` only. $\square$

Corollary 4.3 (Exact sentinel; fail-low certifies non-capturability). *At any king-capturable node (depth ≥ 1 , in-band window) the search reports exactly M_U ; contrapositively, any in-band report $< M_U$ — in particular any fail low — certifies that no king capture was available. This is what makes the depth-0 stalemate probe of Section 3 meaningful.*

Three conditions are load-bearing, and each is an edit a well-meaning PR could make:

- **Position-exact keying.** Every store into `tp_move[p]` happens inside a call at p itself; a ply-shared killer table (a common engine idiom) would let a quiet killer harvested at one position cut off at a different, king-capturable one, breaking the induction.
- **Captures sort first.** King captures have value $\geq M_L$, strictly above every quiet move.
- **In-band windows.** An out-of-band $\gamma > M_U$ could let a quiet move cut off (and be stored) above the capture’s sentinel.

The residual exception is the null-move yield: it stores nothing (so `KillerOK` survives) but can end the loop below the sentinel; sunfish guards it only by `|pos.score| < 500`, a guard its own `FIXME` concedes is heuristic. The condition under which the guard closes the hole is the named hypothesis (`NullGuardBlocksAtCaptures`, `Killer.lean`).

Empirical companion. Before the proof existed, the conjecture was checked empirically on the production engine: **0 violations in 694,533 killer cutoffs over 1,270 real-game positions**. The proof upgraded “empirically absent” to “impossible given in-band windows and value ordering” — and, usefully, told us exactly which future edits would void the guarantee.

5 A Soundness Taxonomy for Selective Search

The campaign’s conceptual yield is a classification of how a selective search may consult its window γ . Sunfish’s maintainer stated it as a principle, which we quote verbatim:

“ γ may shape termination, and may trigger shortcuts whose value provably one-side-bounds the same function; it must never select between incomparable evaluations.”

The three classes, each with its canonical citizen:

5.1 Class (a): γ -shaped termination — always sound

The beta cutoff itself: γ decides only *when the loop stops*, and fail-soft reporting keeps every report a bound on the same negamax. This is Theorem 2.2; nothing further is needed. Null-window search, iterative deepening, and aspiration re-probing all live here.

5.2 Class (b): γ -triggered shortcuts that one-side-bound the same function — sound

Futility pruning is the canonical citizen. At depth ≤ 1 , once $\text{pos.score} + \text{val}(m) < \gamma$, sunfish answers with the *static estimate* $\text{pos.score} + \text{val}(m)$ instead of searching the child. The estimate is below γ , so it is only ever a fail-low report, and BoundSpec demands of a fail-low report exactly one inequality: $-\text{negamax}_d(m) \leq \text{pos.score} + \text{val}(m)$.

We initially stated that inequality as a hypothesis (*FutilityOK*). It was then *discharged*: sunfish builds child positions incrementally, and the score identity

$$\text{eval}(m) = -(\text{eval}(p) + \text{val}(p, m)) \quad (m \in \text{moves}(p))$$

is literal in `Position.move`. With the identity as a structural property of the model (`ValGame`, `Tricks.lean`), the futility test $\text{pos.score} + \text{val} < \gamma$ is integer-equivalent to “the child’s stand-pat meets its window $1 - \gamma$ ”, at depth 0 the stand-pat is yielded first and fails high immediately, and fail-soft returns exactly the child’s score:

Theorem 5.1 (*FutilityOK discharged — as an equality*). *In any game with the score identity, for every position p and move $m \in \text{moves}(p)$,*

$$-\text{negamax}_0(m) = \text{eval}(p) + \text{val}(p, m).$$

The futility yield is not an estimate: it equals the depth-0 child search it replaces. (`futilityOK_discharged`, `Tricks.lean`)

Theorem 5.2 (*Futility-pruned search satisfies BoundSpec*). *For any game with the score identity, under in-band windows and the remaining hypothesis FutilityMateOK (a move valued as a king capture, $\text{val} \geq M_L$, really wins: the fail-high `else MATE_UPPER` bypass), the futility-pruned search satisfies BoundSpec against plain negamax — a single value function, no interval. (`boundFut_spec`, `Tricks.lean`)*

Proof sketch. At depth 1 (the futility zone) each per-move yield is analyzed by cases: a futile quiet move yields exactly $-\text{eval}(m)$ by Theorem 5.1; a futile king capture yields the exact sentinel, a fail high justified by *FutilityMateOK* and the in-band window; a non-futile move is an

exact depth-0 child. A member-restricted variant of the loop lemma (`searchMoves_spec_mem`, `Tricks.lean`) (the score identity only speaks about legal moves) closes the loop; deeper depths are Theorem 2.2’s argument verbatim. Sorry-free. $\square$

Fine print made explicit by the model: the *quantified* FutilityOK (over all depths) is not dischargeable — plain negamax at $d \geq 1$ has no stand-pat option — but the search only ever consumes the $d = 0$ instance, and that instance is the score identity on the nose. The original statement over-required.

5.3 Class (c): γ -selected incomparable evaluations — unsound in the point sense

Late Move Reductions is the canonical citizen. A late quiet move at depth ≥ 3 is probed at depth -2 ; a reduced fail low is yielded as-is, a reduced fail high is re-searched at full depth and only that result is yielded. Here γ *selects which of two incomparable value functions the yield is a fact about*, and the docstring becomes unprovable as stated: no single value function backs both sides.

The honest replacement is an interval. Define the mutually recursive pair $(V^{\text{hi}}, V^{\text{lo}})$ with side alternation:

$$\begin{aligned} V_d^{\text{hi}}(p) &= \max_m -V_{d-1}^{\text{lo}}(m) && \text{(all full depth)} \\ V_d^{\text{lo}}(p) &= \max_m \begin{cases} \min(-V_{d-2}^{\text{hi}}(m), -V_{d-1}^{\text{hi}}(m)) & \text{if } m \text{ reducible} \\ -V_{d-1}^{\text{hi}}(m) & \text{otherwise} \end{cases} \end{aligned}$$

(`lmrVal`, `Lmr.lean`), over an abstract reducibility predicate $\text{red}(d, i, m)$ generalizing sunfish’s depth ≥ 3 and `i_m` ≥ 5 and `val` $< \text{QS}$.

Theorem 5.3 (The LMR interval spec). *For every depth, position and window: a fail-high report of the LMR search is $\leq V_d^{\text{hi}}(p)$, and a fail-low report is $\geq V_d^{\text{lo}}(p)$. (`boundLmr_spec`, `Lmr.lean`)*

Theorem 5.4 (The sandwich). *$V_d^{\text{lo}}(p) \leq \text{negamax}_d(p) \leq V_d^{\text{hi}}(p)$ pointwise: LMR reports still confine the full-depth value — but only from the side that happened to be reported, and against the widened pair, not the point. (`lmrVal_sandwich`, `Lmr.lean`)*

Proof sketch of Theorems 5.3 and 5.4. Strong induction on depth via a two-target generalization of the loop lemma (`searchMovesIdx_spec`, `Lmr.lean`): each move’s report is bounded above by one weight function when it fails high and below by another when it fails low, and the loop’s fail-soft result inherits both bounds against the respective folds. The per-move case analysis follows the merged code: a reduced fail low yields a shallow fact (the child’s V^{hi} at the reduced depth bounds it, whence the min); a reduced fail high falls through to the full-depth re-search. The sandwich is a separate induction using monotonicity of the indexed fold. Both sorry-free. $\square$

Two facts about this modeling were *surprises* — both were claims the authors believed and the proof assistant refused; we recount them honestly in Section 9.

The practical consequence is a transposition-table pathology:

Theorem 5.5 (No crossing without LMR). *For the unreduced search, a fail-high report can never exceed a fail-low report at the same (pos, depth): both bracket $\text{negamax}_d(p)$, so a `tp_score` entry always satisfies lower $\leq$ upper. (`bound_no_crossing`, `Lmr.lean`)*

Counterexample 5.6 (TT crossing under LMR, machine-checked). There is a three-position game and windows $\gamma = -60, 0$ on which the LMR search at the same (pos, depth) produces a fail-high report of +10 and a fail-low report of -50: a `tp_score` entry with *lower* = 10 > -50 = *upper*. The single root move looks bad shallow but is good deep; probing $\gamma = 0$ takes the reduced branch and fails low at -50, probing $\gamma = -60$ bypasses it, re-searches at full depth and fails high at +10. (`lmr_tt_crossing`, `Lmr.lean`)

MTD-bi’s bracket $lower \leq score \leq upper$ is therefore conditional on the $V^{hi} - V^{lo}$ gap staying within what `EVAL_ROUGHNESS` and re-probing absorb. On the production engine the crossing is real but rare: **0.016% of nodes** measured over real games. The mitigation is two lines — on store, widen the stale side of the entry instead of asserting a contradiction — and measured ELO-neutral at $+12 \pm 28$ (i.e., free). Killer and stalemate lemmas are unaffected: king captures have $val \geq M_L > QS = 40$ and are never reducible (`RedRespectsCaptures`, `Lmr.lean`); the killer is pre-loop and structurally never reduced; reduced fail lows are $< \gamma$ and never reach the `tp_move` store; and the $-M_U$ king-loss sentinel is depth-independent, so stalemate detection still sees exact sentinels from reduced searches.

6 The Ledger of Bets: Named Hypotheses

Every pruning trick that is only conditionally sound is kept out of the proven core and given a *named hypothesis*: the condition, not the induction, is the interesting content. The discipline this buys is a repository rule: **a search-changing PR must identify which lemma it preserves, weakens, or newly depends on** (see `formal/README.md` for the full PR-guideline ledger). Every unproven assumption is named, stated, and where possible measured.

Hypothesis	Statement (informal)	Status
NullOK	In every non-terminal position some legal move scores at least the pass value. <i>Zugzwang is precisely its negation</i> ; hence sunfish’s $ \text{score} < 500$ guard.	Named; <code>boundNull_spec</code> sketched
TableOK	Every stored <i>(lower, upper)</i> brackets $\text{negamax}_d(p)$; requires the <i>Bounded</i> side condition, without which the fresh entry $(-M_U, M_U)$ is not a valid bracket and the theorem is false.	Named; <code>boundTT_spec</code> sketched
FutilityOK	Static estimate dominates the true child value.	Discharged (Thm. 5.1)
FutilityMateOK	A move valued as a king capture really wins everything.	Named (a fact about chess, not arithmetic)
MateValuesAreKingCaptures	Mate-band values come from real king captures (Hyp. 3.3).	Named; necessity machine-checked (Cex. 3.5)
CheckProbeOK	The depth-0 null probe decides check.	Named; true because QS never prunes king
MateDepthMonotone	A mate-band lower bound survives one more ply; lifts to arbitrary deeper service (<code>mateEntry_deep_service</code> , <code>Tricks.lean</code>).	Named; lift proven
MateDepthStable	Mate-band membership is depth-independent for $d \geq 1$; what <i>shallower</i> service of sentinel mate entries assumes. Any variant using it should document itself as weakening <code>BoundSpec</code> to mate-band membership.	Named; derivation from <code>KingGoneStable</code> sk
KingGoneStable	A captured king is permanent, on both sides of the sign alternation.	Named
NullGuardBlocksAtCaptures	The null-move guard fires at no king-capturable position: closes the residual killer exception (Sec. 4).	Named
QuietEvalsInBand	Quiet positions evaluate strictly inside the mate band.	Named
RedRespectsCaptures	Reductions never touch king captures ($\text{val} \geq M_L > QS$).	True by construction in sunfish
ExtKeyIndependent	Extended-search value is a function of $(\text{pos}, \text{depth})$ alone.	Refuted (<code>extended_value_not_key_indepe</code>)

Two entries deserve a sentence each. `ExtKeyIndependent` is stated as the *false* claim a $(\text{pos}, \text{depth})$ -keyed table silently makes once search extensions depend on history (e.g. a recapture extension keyed on the last-capture square); a two-state counterexample refutes it, and the consequence is that such a TT key must include the history component — sunfish’s `can_null` flag being the same phenomenon in miniature, duly part of its key. And `soften_null_window` (`soften_null_window`, `Tricks.lean`) is a small proven lemma: for windows strictly above the mate band, mate-distance softening does not disturb null-window fail-high decisions.

7 Empirical Methodology and Honest Numbers

Functional tests tell you the engine is correct; they tell you nothing about whether a change makes it stronger. Every strength-relevant change in this campaign was priced by a tournament between engines frozen with `git archive` (code *and* tools — no shared files), playing color-swapped pairs from a random-order opening book under fastchess [2], typically 300 games at `tc=4+0.04` ($\approx \pm 30$ ELO at 95%), with adjudication, on an otherwise idle machine. The full methodology, each rule with its origin story, is `docs/TESTING.md` in the repository.

7.1 Results

Change	ELO	Disposition
Late Move Reductions	$+49 \pm 34$	merged (LOS 99.8%, 300 games)
TT store-clamp (crossing mitigation)	$+12 \pm 28$	merged (free; principled)
Deterministic LMR (no re-search)	-48 ± 33	declined
Conservative LMR	-16 ± 38	declined
Sticky-entry consistency restoration	-33	declined
Ordering-only consistency restoration	-34	declined
Null-move soundness completion	-28	declined
Reverse futility	$+45 \rightarrow -1$	artifact (time regime); declined
Mate-score softening	$+19 \rightarrow -22$	artifact (time regime); declined

Table 1: Measured prices. Error bars are 95% where quoted; all matches per `docs/TESTING.md` (300 games, `tc=4+0.04`, opening book, adjudication) unless noted. The middle block prices *consistency*: every attempt to restore point-soundness under LMR — making the reduced search deterministic, keeping entries sticky, or restoring ordering only — costs 16–48 ELO. The proofs say what is being traded; these numbers say what the trade is worth.

Three narrative points, because the numbers alone mislead:

The price of point-consistency. Theorem 5.3 says LMR is sound only as an interval. One can buy back the point spec — and each way of doing so was priced: deterministic LMR (-48 ± 33), a conservative reduction condition (-16 ± 38), sticky TT entries (-33), ordering-only (-34). Every restoration is ELO-negative. Likewise, completing null-move soundness (searching the pass at a depth for which the spec discharges) was priced at -28 and declined: the engine keeps the bet, and the ledger keeps its name (NullOK).

Time-regime artifacts. An earlier survey measured reverse futility at $+45$ and mate-score softening at $+19$. After a fix to time handling (in-search deadlines replacing between-iteration checks), both were re-measured on the fixed baseline: -1 and -22 . The original numbers were artifacts of the old time regime, not chess. Rule: after any time-handling change, re-baseline everything you believed.

The multi-TC lesson. An opening think-time ramp (cap of $0.1s \times$ ply) shipped with no cutoff and silently governed *entire games* at long time controls: at $15+3$ the proper budget is $\sim 25s$ /move but the ramp still bound at move 60, and a user’s game ended in mate with 14

of 15 minutes unused. The bug was invisible at the 4+0.04 test TC, where the ramp stops binding at ply 2 — it sailed through 300-game matches. It was caught by a user report and fixed the same day; the fix was verified by driving the per-move time curve directly at 15+3 (17–24s observed mid-game vs. 3–9s before). This became a standing rule: time-management changes must be validated at multiple time controls, including a direct per-move time-curve check, because fast-TC matches are structurally blind to long-TC time bugs.

Contamination stories, briefly. A contaminated run (engines sharing a live checkout) once measured -60 ELO at LOS 0.02% for a change later shown to search bit-identical node counts — hence the freeze rule and the old-vs-old control match. Tournament managers reuse engine processes across games, so state surviving `ucinegame` acts as invisible cross-game learning: a four-way eviction-policy round-robin ranked $+66 / +47 / +15 / -132$ across candidates whose *per-game* search was in some cases provably identical. Behavior-neutrality is checked by depth-limited lockstep (moves *and* node counts), never by clock.

7.2 The frame

Proofs identify the trade; tournaments price it. Neither loop alone would have shipped LMR correctly: without the proof we would not have known to clamp the TT store (or that the killer and stalemate arguments survive reduction); without the tournaments we would have “fixed” the inconsistency at a cost of tens of ELO.

8 Related Work

Nipkow’s invited ITP 2024 paper [7] and the accompanying Isabelle/HOL AFP entry [6] verify fail-hard and fail-soft alpha-beta over linear orders, distributive lattices and de Morgan domains; the fail-soft correctness statement there is the two-sided bound that our BoundSpec specializes to a null window, and our loop invariant (“the running best is a fail-soft report of the fold so far”) follows the same shape. *Formal Verification of Minimax Algorithms* [5] verifies minimax variants with alpha-beta and transposition tables in Dafny — the closest prior treatment of the table invariant (our `TableOK`). Knuth and Moore’s classical analysis [4] and the MTD(f) framework [8] supply the algorithmic background; futility pruning in the form sunfish uses traces to Heinz [3].

Our position: to our knowledge this is the first Lean 4 formalization of null-window/MTD-bi search, and the first line of work to pair machine proofs of search soundness with ELO-measured prices on a production engine — one that plays rated games in public. We found no prior formal treatment of king-capture (pseudo-legal) search, of a stalemate correction, of killer-table invariants, or of an interval-valued specification for Late Move Reductions.

9 Lessons and Threats to Validity

The model–code gap. All theorems are about a Lean model, not about `sunfish.py`. The gap is managed, not closed: the model is line-mapped (the correspondence table in `formal/README.md` pairs every Lean definition with the sunfish lines it transcribes), the counterexamples are constructed to exercise exactly the modeled mechanism, and features excluded from a given model are excluded *explicitly* with a stated reason (e.g. table eviction only forgets entries, which trivially preserves `TableOK`). Still, quiescence is collapsed into `eval`, move ordering is modeled by

its load-bearing consequence only, and the null-move depth reduction (depth -3) is deliberately unmodeled. A reader should trust the theorems as statements about the mechanisms, not as end-to-end verification of the Python.

Two author claims refuted by the proof assistant, mid-project. We record these because they are the strongest evidence in this paper that the machine was doing work the authors could not do by inspection. (1) *The fail-high-vs- V_{full} error.* While specifying LMR we claimed that fail-high reports remain sound against full negamax — the re-search guard, we reasoned, ensures no reduced fail high is ever yielded. Lean refused the induction: the guard protects only the *immediately* reduced move, while a parent fail high is certified by child fail lows, which are only sound against the children’s *reduced* values — the contamination is recursive and cannot be scrubbed at one level. This is why V^{hi} ’s children are V^{lo} -valued (Theorem 5.3) and why the mutually recursive pair exists at all. (2) *The V_{reduced} target error.* We claimed the folklore fail-low target — reducible moves simply valued one ply shallower (negamaxShallow, kept in the development for the record) — was the right lower target. Also false: on the re-search fall-through, the yielded value bounds the *deep* child value while the shallow value has just failed high and can exceed everything yielded; the sound per-move entry is the *minimum* of the two depths. Both errors would have survived code review; neither survived `lake build`.

Measurement contamination. Every rule in `docs/TESTING.md` exists because its absence produced a confident wrong number in this repository: the -60 ELO phantom regression (shared checkout), the $\pm$ dozens swings from cross-game table persistence (process reuse), fictional error bars from deterministic engines without opening books, and the long-TC time bug invisible at fast TC. We treat the methodology document as part of the scientific apparatus, on the same footing as the proofs.

Threats. Single engine, single day, one maintainer’s conjectures; ELO error bars at 300 games are ± 30 -ish and several middle-block numbers in Table 1 are single runs; the taxonomy of Section 5 is validated on one engine family (null-window MTD-bi) and its transfer to PVS/aspiration engines is argued, not proven.

10 Availability

Everything is in the public sunfish repository [1]: the engine (`sunfish.py`), the Lean development (`formal/`, building with `lake build` from a pinned Lean 4.12.0 toolchain, zero dependencies beyond core Lean and the `omega` tactic — no Mathlib — so a full build takes seconds), the methodology (`docs/TESTING.md`), and the lemma inventory, correspondence tables and PR-guideline ledger (`formal/README.md`). CI builds the proofs and plays real protocol games: fastchess tournaments and a mock-Lichess integration test that drives the engine over the wire exactly as the production bot is driven. The engine itself plays continuously on Lichess as `sunfish-engine`.

Acknowledgments.

References

- [1] Thomas Dybdahl Ahle and contributors. Sunfish: a python chess engine in 131 lines of code. <https://github.com/thomasahle/sunfish>, 2026. Formal development in `formal/`; plays on Lichess as `sunfish-engine`.
- [2] Disservin and contributors. fastchess: a command-line tool for managing chess engine tournaments. <https://github.com/Disservin/fastchess>, 2025.
- [3] Ernst A. Heinz. Extended futlity pruning. *ICCA Journal*, 21(2):75–83, 1998.
- [4] Donald E. Knuth and Ronald W. Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4):293–326, 1975. doi:10.1016/0004-3702(75)90019-3.
- [5] Formal verification of minimax algorithms. arXiv preprint, 2025. Verifies minimax variants with alpha-beta pruning and transposition tables in Dafny. <https://arxiv.org/abs/2509.20138>. TODO: fill in the author list from the arXiv page. arXiv:2509.20138.
- [6] Tobias Nipkow. Alpha-beta pruning. *Archive of Formal Proofs*, 2024. https://isa-afp.org/entries/Alpha_Beta_Pruning.html, Formal proof development.
- [7] Tobias Nipkow. Alpha-beta pruning verified. In *15th International Conference on Interactive Theorem Proving (ITP 2024)*, volume 309 of *LIPICs*, pages 1:1–1:18. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024. Invited talk. doi:10.4230/LIPICs.ITP.2024.1.
- [8] Aske Plaat, Jonathan Schaeffer, Wim Pijls, and Arie de Bruin. Best-first fixed-depth minimax algorithms. *Artificial Intelligence*, 87(1–2):255–293, 1996. doi:10.1016/0004-3702(95)00126-3.